SQL Server 2025 Vectors

Under the Hood and Into Production

Anthony Nocentino · Level 300 · Database Management



Agenda

60 minutes of content, then 15 for questions. Context first, then the demo that proves it.



0:00	Vectors 101: the math you need	5 min
0:05	Inside the VECTOR type	3 min
0:08	**Demo 1: vector internals**	12 min
0:20	Vector indexes: where they help and where they don't	5 min
0:25	**Demo 2: index performance at scale**	15 min
0:40	Into production: the lab and the app	3 min
0:43	**Demo 3: the RAG application**	15 min
0:58	Takeaways, then questions	2 + 15 min

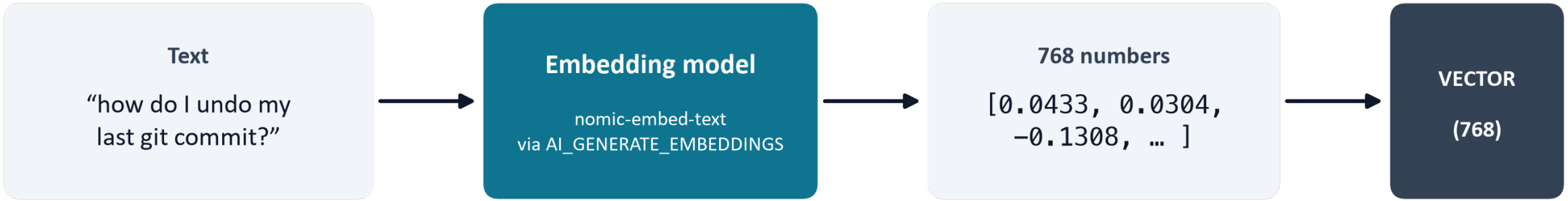
Vectors 101

The math you need for the next hour, and nothing more



From text to a vector

Meaning in, a fixed-length list of numbers out



The model decides the dimension count. Similar meaning lands close together:

“git revert last commit…” distance 0.12 **“quattro formaggi pizza” distance 0.68**

The math in three ideas

Everything in this session is built from these

A vector is a list of numbers

VECTOR(3) '[1, 0, 0]'. Each number is a coordinate. Together they give a direction and a length.

- Our model: 768 numbers
- Same model, same count, always

Length is the norm

$|v| = \sqrt{(v_1^2 + v_2^2 + \ldots + v_n^2)}$

- [3, 4, 0] has length 5
- A unit vector has length 1

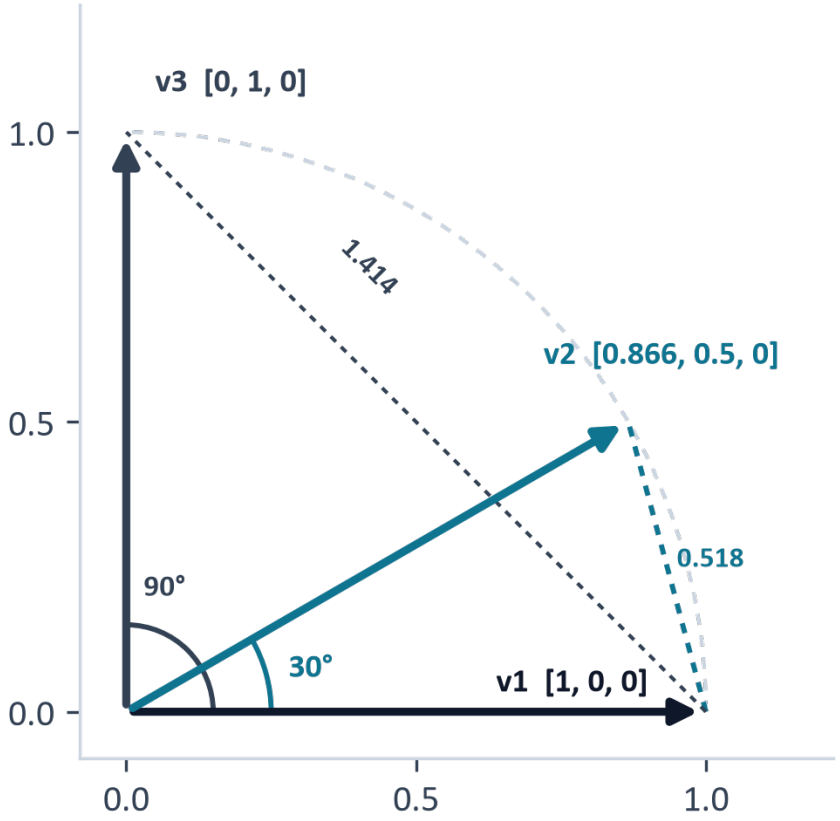
The dot product measures alignment

$v \cdot w = v_1w_1 + v_2w_2 + \ldots = |v|\ |w| \cos \theta$

- Same direction: largest
- At right angles: 0

Three vectors, three distances

VECTOR_DISTANCE: lower always means closer



Pair	cosine	euclidean	dot
v1 vs v1 (same)	0	0	−1
v1 vs v2 (30°)	0.134	0.518	−0.866
v1 vs v3 (90°)	1	1.414	0

cosine = 1 − cos θ: only the angle counts

euclidean = the straight-line gap between the tips

dot = −(v · w), negated so lower is closer

Unit vectors, so dot = cosine − 1, and euclidean = √(2 × cosine)

Cosine, Euclidean, or dot?

On unit vectors all three rank results the same. They only disagree when length varies.

Cosine

Use when only meaning matters.

- The default for text embeddings
- Ignores length: short and long text on one topic score alike
- Readable: 0 = same direction, 2 = opposite

Dot

Use when vectors are unit length, or length should win.

- Same order as cosine on unit vectors, and cheaper
- Unnormalized: longer vectors score closer
- Recommenders, where length is popularity

Euclidean

Use when absolute position matters.

- Images, sensors, clustering
- Unnormalized: a length gap counts as distance
- On unit vectors: √(2 × cosine), same order

Rule of thumb: use the metric your embedding model's documentation recommends, and build the index with the same one. VECTOR_SEARCH only uses an index whose metric matches the query.

Normalize, or keep the magnitude?

Our model's vectors measure 1.000 long, so cosine and dot rank every result the same

Normalize to unit length when

Meaning lives in the direction, not the length.

- Text and most embedding models
- You want dot speed with cosine results
- You mix sources that produce different lengths

Keep the magnitude when

Length is part of the signal.

- Counts, intensity, popularity, ratings
- The model was trained for raw dot or Euclidean
- You need distances in real units

SIMILARITY SEARCH

Embed the question. Measure it against the stored vectors. Keep the closest K.

On this model, a strong match scores about 0.12 and an unrelated one about 0.68.

Inside the VECTOR type

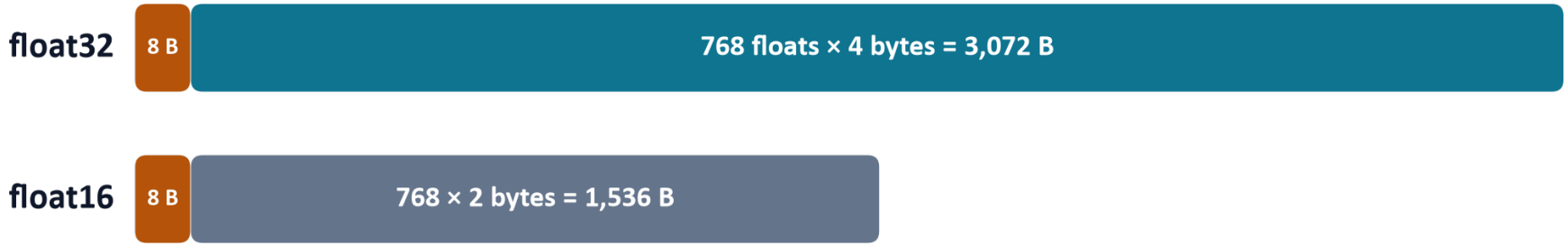
Storage layout, precision, and exact search



Anatomy of a VECTOR

IEEE 754 floats behind an 8-byte header, stored in the row

One VECTOR(768) value: an 8-byte header, then the floats



One element, bit by bit (IEEE 754, little-endian on disk)



Dimensions set the size

VECTOR(n) stores n numbers behind the same 8-byte header

bytes = 8 + n × 4 float32 **bytes = 8 + n × 2** float16

Embedding model	Dimensions	VECTOR(n) float32	VECTOR(n, float16)
nomic-embed-text (our demos)	768	3,080 B	1,544 B
mxbai-embed-large	1,024	4,104 B	2,056 B
OpenAI text-embedding-3-small	1,536	6,152 B	3,080 B
OpenAI text-embedding-3-large	3,072	won't fit (Msg 2717)	6,152 B
The limit	1,998 / 3,996	8,000 B	8,000 B

Measured with DATALENGTH on SQL Server 2025 CU10. Size grows in a straight line with dimensions.

float32 or float16?

Use float16 when storage or dimensions are the limit and your vectors are unit length. Otherwise float32.

float32 (the default)

4 bytes per element, up to 1,998 dimensions.

- Generally available in SQL Server 2025
- Full precision for any workload
- Every client driver handles it

float16 (preview)

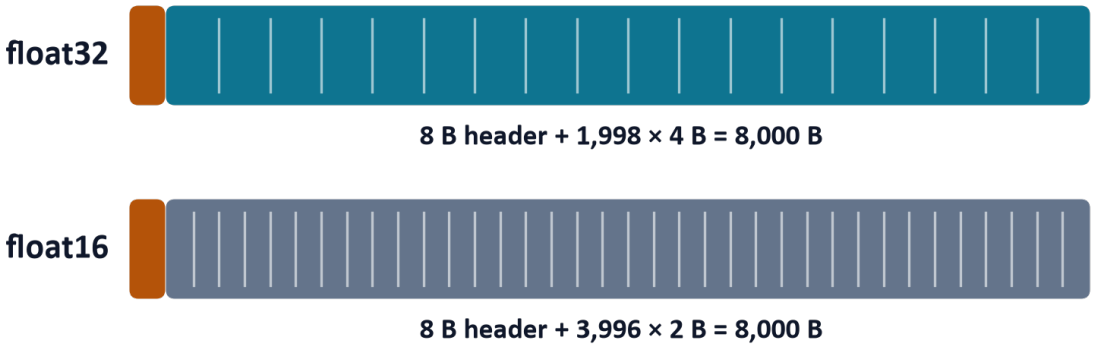
2 bytes per element, up to 3,996 dimensions.

- Half the storage and I/O; distances moved ~0.00001
- Fits 3,072-d models like text-embedding-3-large
- Needs PREVIEW_FEATURES; binary transport needs ODBC 18.7.1+ or JDBC 13.4+
- Can't mix with float32 in VECTOR_DISTANCE

Why float16 fits twice the dimensions

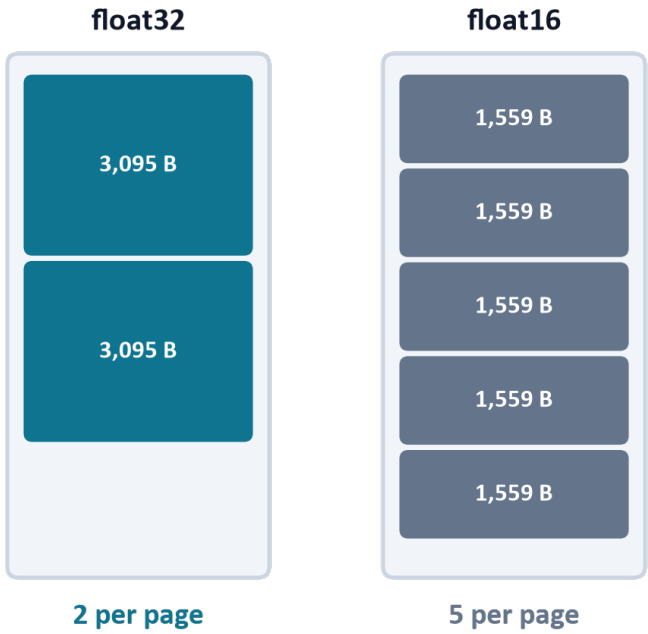
The cap is 8,000 bytes per value, not a number of dimensions

Every VECTOR value is capped at 8,000 bytes



Same bytes. Half the size per number. Twice the numbers.

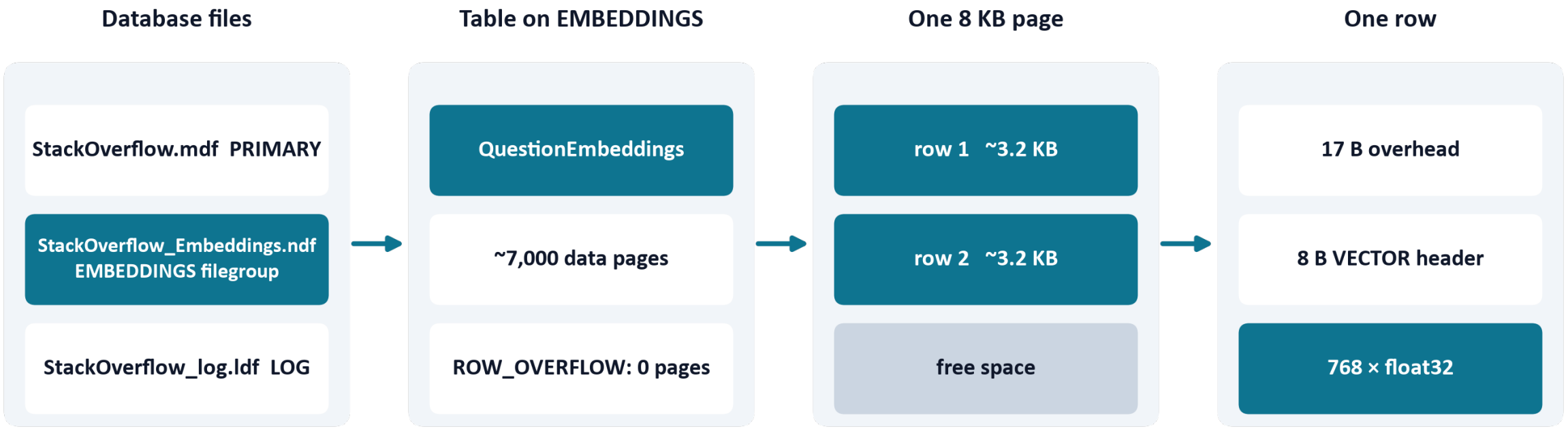
768 dims on one 8 KB page



Same page, more vectors: 1,000 vectors at 768 dims took 500 pages as float32 and 200 as float16. An exact scan reads 60% fewer pages.

Where the bytes live

From database file, to filegroup, to an 8 KB page, to the row



~1.5 rows per page, so an exact scan of 10,000 vectors reads about 7,000 pages.

Decode those 3,072 bytes by hand and all 768 floats match what SELECT prints.

What vectors cost you

Measured on the demo stack. Plan around these.

~15 ms*

per embedding call

- One HTTPS call per row, no batching
- 1M rows: about 4 hours here

3.6 KB

of log per inserted row

- 1M rows: about 3.6 GB of log
- Size the log for loads and rebuilds

1.5 rows

per 8 KB page

- A 3 KB vector stays in the row
- 10K vectors: ~7,000 pages to scan

3–10×

slower on a cold cache

- Exact scans read ahead in big chunks
- Storage speed matters

** Environment specific: nomic-embed-text on Ollama, Apple M2 GPU, through a local NGINX TLS proxy. Your model, hardware, and network will differ, so measure it.*

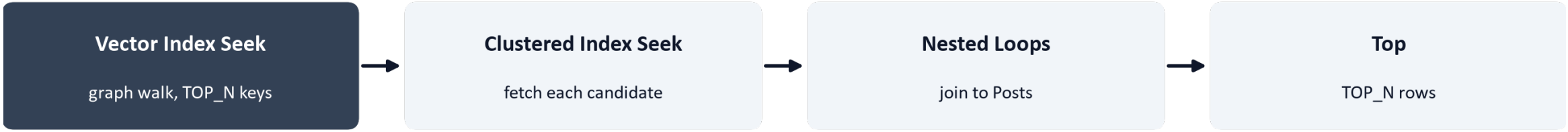
How SQL Server runs a similarity search

No B-tree helps: a vector can't be an index key (Msg 1978) or compared with = (Msg 8117)

Exact kNN (no vector index)



ANN (VECTOR_SEARCH with a DiskANN index)

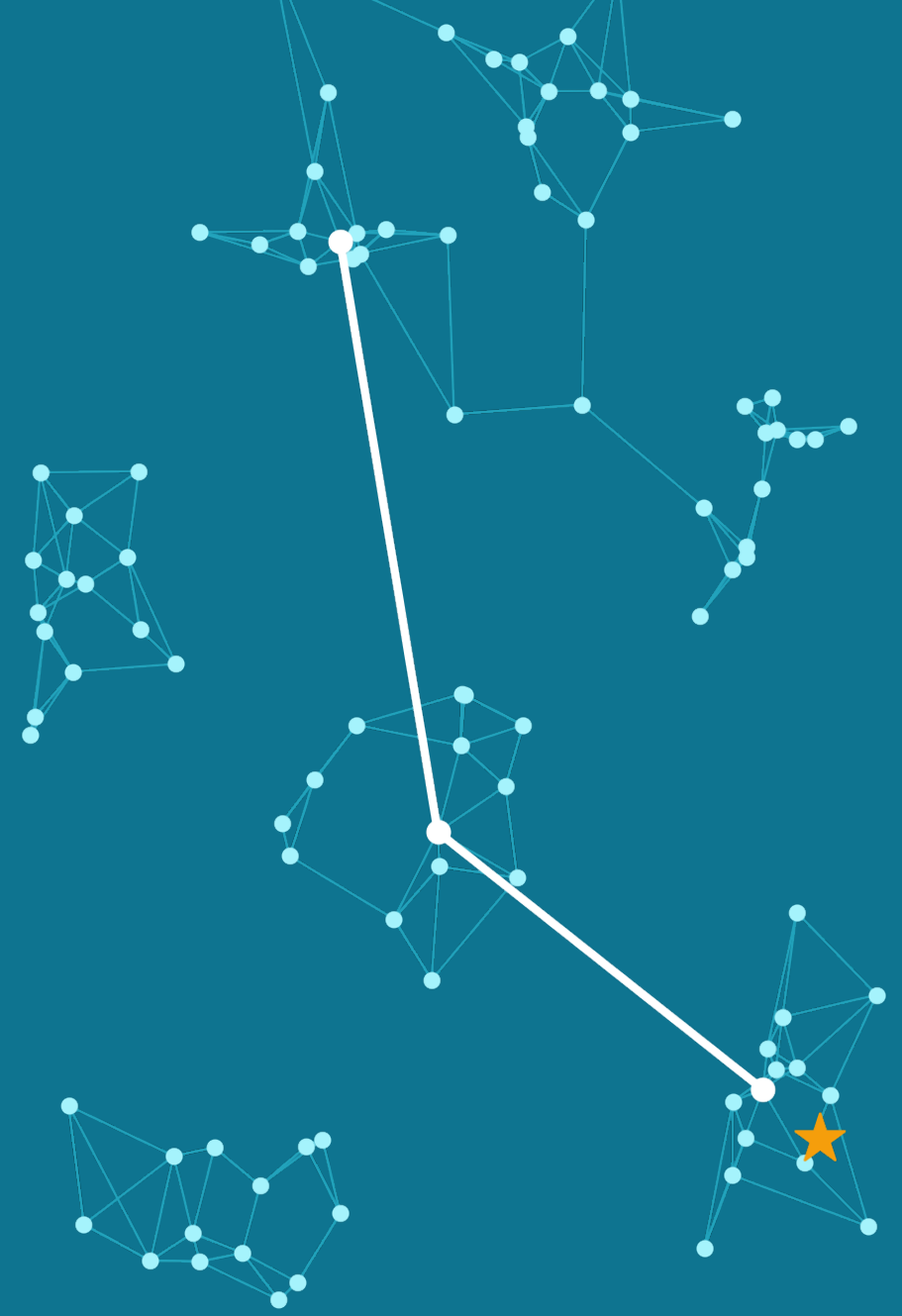


Data flows left to right here (SSMS draws it right to left). The index replaces the scan and the per-row math. Estimated rows on Vector Index Seek show 1.

LIVE DEMO

Demo 1: Vector internals

Files and pages · decode a vector off the page · float16 · distance · the exact-search plan



Vector indexes

Where they help, and where they don't



Exact kNN vs approximate ANN

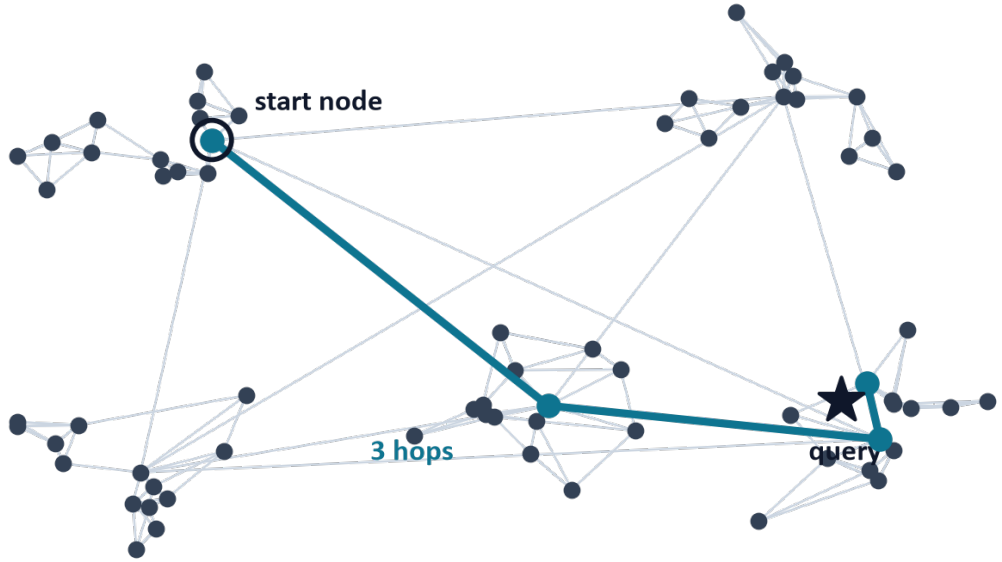
Exact visits every vector. ANN walks a graph and visits a few.

Exact kNN: measure every vector



Every row, every time. O(n). Perfect recall.

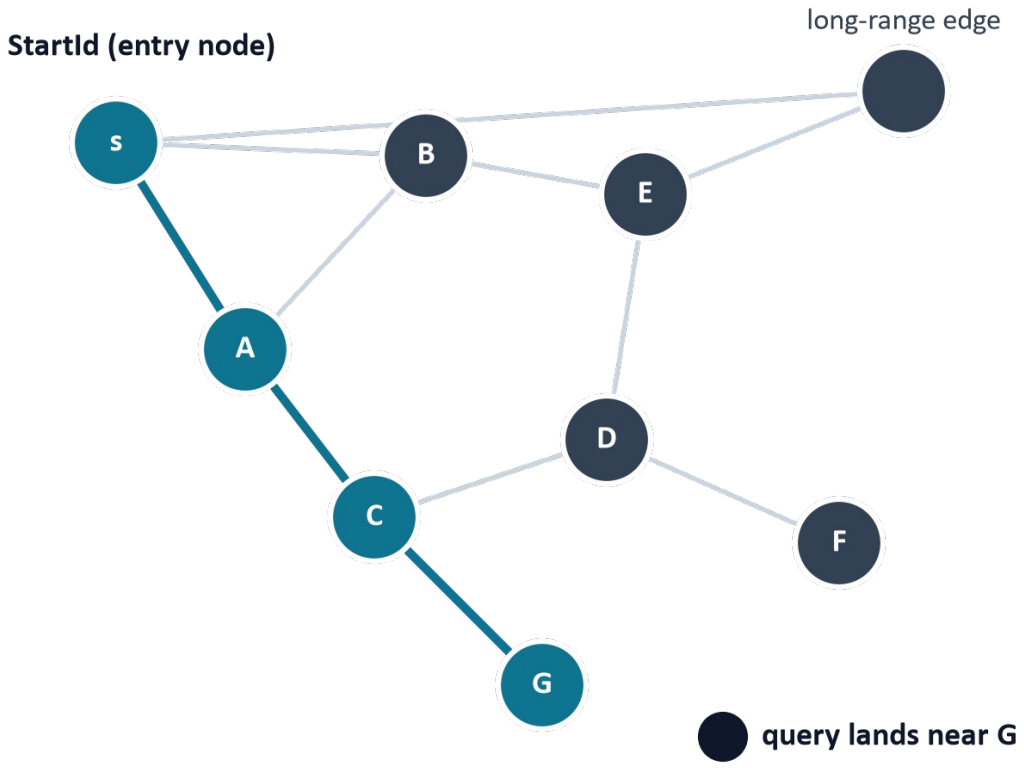
ANN (DiskANN): walk a graph



A few hops toward the query. Far fewer reads. Approximate.

Inside DiskANN

A graph built once over every vector, walked greedily for each query



Greedy search

Start at the entry node, hop to the neighbor closest to the query, stop when nothing is closer.

R = 48, L = 48

At most 48 edges per node; a 48-entry candidate list during the build. Read from sys.vector_indexes.

Stored beside the table

Graph edges live in an internal table: about 3 MB for 10K rows, 261 MB for 1M. The quantization table is empty on SQL Server 2025.

Built, not maintained

CREATE VECTOR INDEX builds the whole graph. 10K rows: 11 to 17 s. 1M rows: 26.5 min.

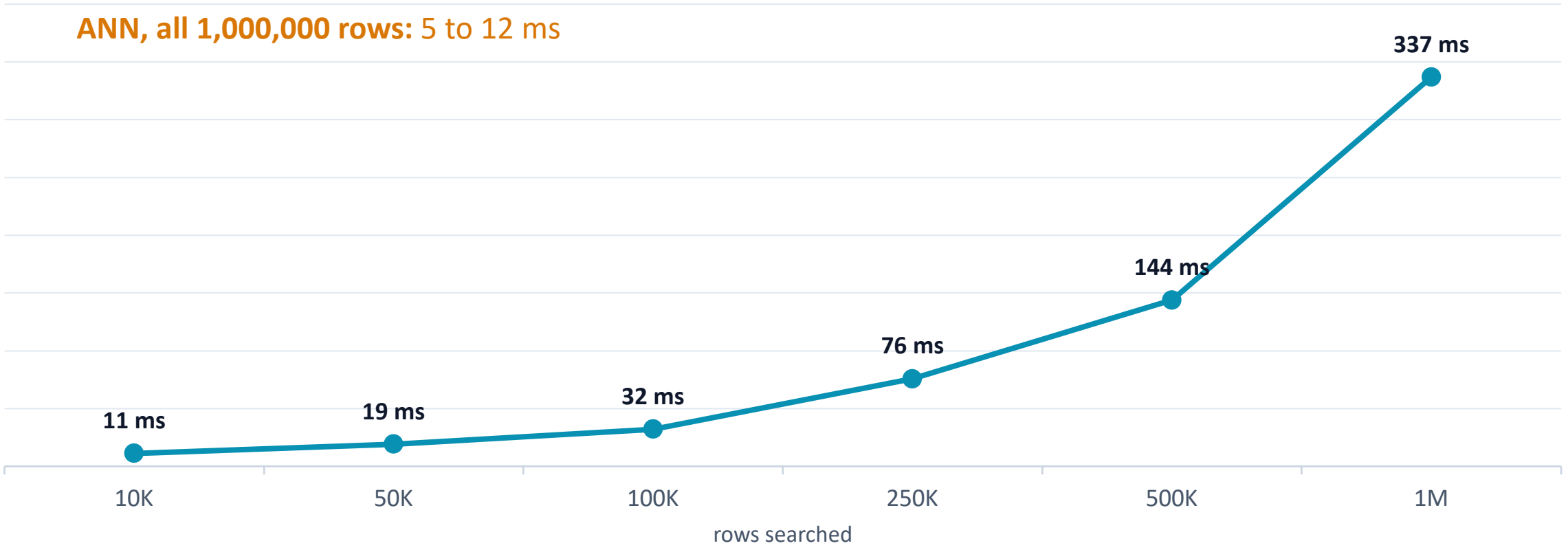
kNN or ANN: which, when

Microsoft's guidance: exact search is fine up to about 50,000 vectors

Situation	Use	Why
Under ~50,000 vectors, or a filter cuts it that far	Exact kNN	Fast enough, perfect recall, nothing to maintain
A small table (about 1,000 rows)	Exact kNN	The index costs more reads than the scan
Hundreds of thousands to millions of vectors	ANN	511,159 vs 7,324 reads for one question at 1M rows
You need every true neighbor	Exact kNN	ANN found 64–86% of the true top 10 at 1M rows
A selective filter, on SQL Server 2025	kNN, or ANN with a large TOP_N	Filters run after the graph walk
The table changes all day	kNN, or ANN on Azure SQL	The 2025 index makes the table read-only

Exact search grows with every row

One exact kNN query on this laptop: average elapsed time by rows searched. Watch the shape, not the ms.



Times are from one laptop and move from run to run. Compare them to each other, not to your servers. Logical reads don't change.

70×

fewer reads with the index

One question over 1,000,000 vectors: 511,159 logical reads for exact kNN, 7,324 with the index. On this laptop, about 160–340 ms against 15 ms.

Recall: the price of approximate

How many of the true top 10 the index found, at 1,000,000 rows

Search	Recall@10	Time per query	Logical reads
ANN, TOP_N = 10	0.64 – 0.86	7 – 24 ms	~5,800
ANN, TOP_N = 50, keep 10	0.94 – 0.99	19 – 43 ms	~13,900
Exact kNN	1.0	160 – 340 ms	511,159

One question, live in Demo 2: “What are the uses for Cross Join?” found 4 of the true top 10 with TOP_N = 10, and all 10 with TOP_N = 50. Recall falls as the table grows (0.96 at 10,000 rows). Ask for more candidates and keep the closest: that's oversampling, and the demo app does it 5×.

Times are from one laptop and move from run to run. Compare them to each other, not to your servers. Logical reads don't change.

Limits on SQL Server 2025, today

All three are fixed in the latest-version index on Azure SQL

Read-only tables

- INSERT, UPDATE, DELETE fail with Msg 42231
- Loading = drop the index, insert, rebuild
- Demo 3 runs that cycle live

Filters run after the walk

- TOP_N = 5 plus a selective filter returned 0 rows
- No error: silent
- Oversample TOP_N to compensate

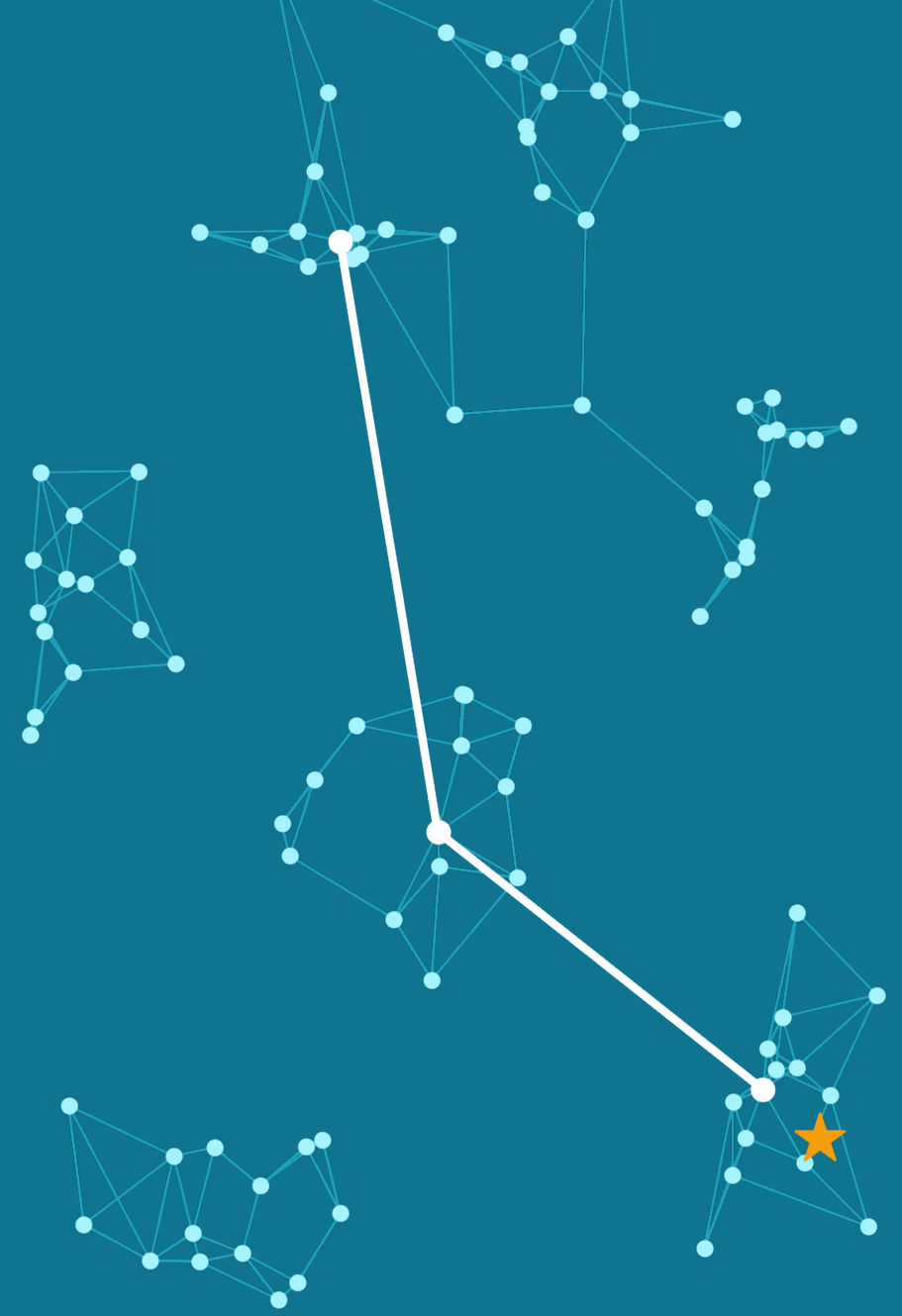
An index version on its way out

- This earlier-version index will be retired
- TOP_N is deprecated
- Latest syntax: SELECT TOP (N) WITH APPROXIMATE

LIVE DEMO

Demo 2: Performance at scale

Build and inspect the index · kNN vs ANN · 1,000,000 rows · recall · the limits



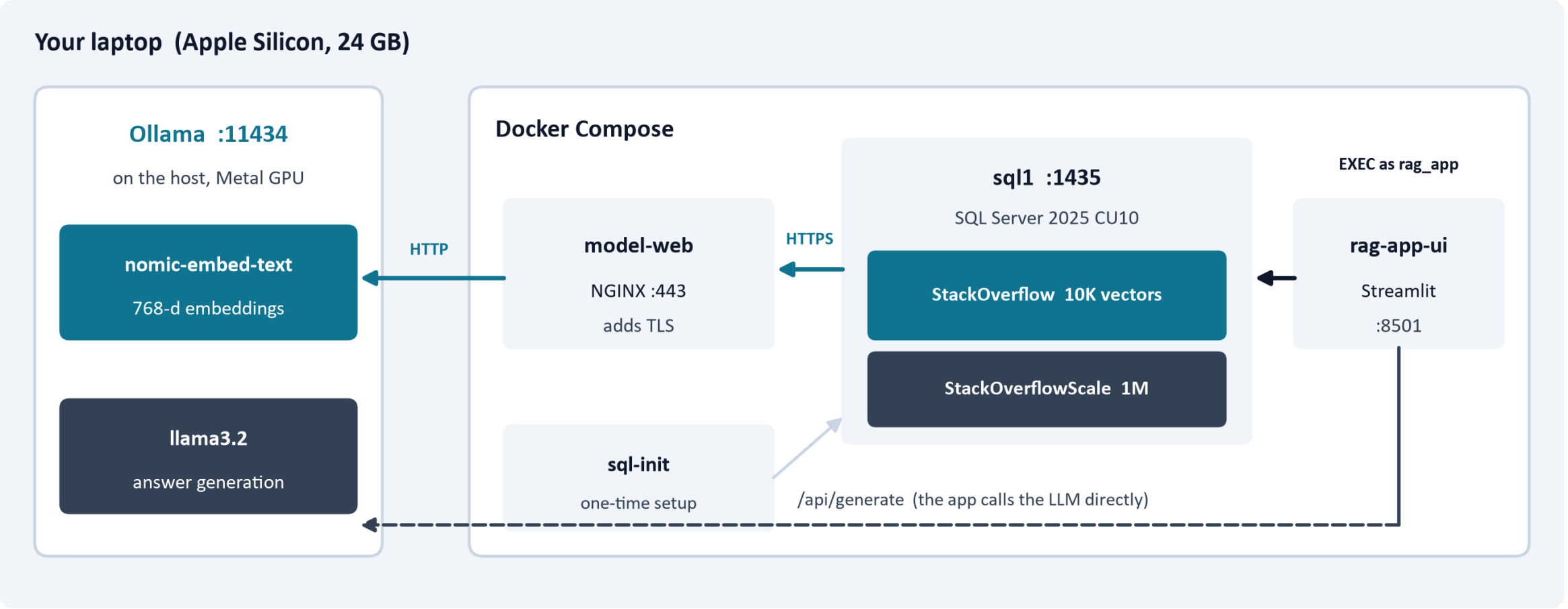
Into production

Build a working application on all of it



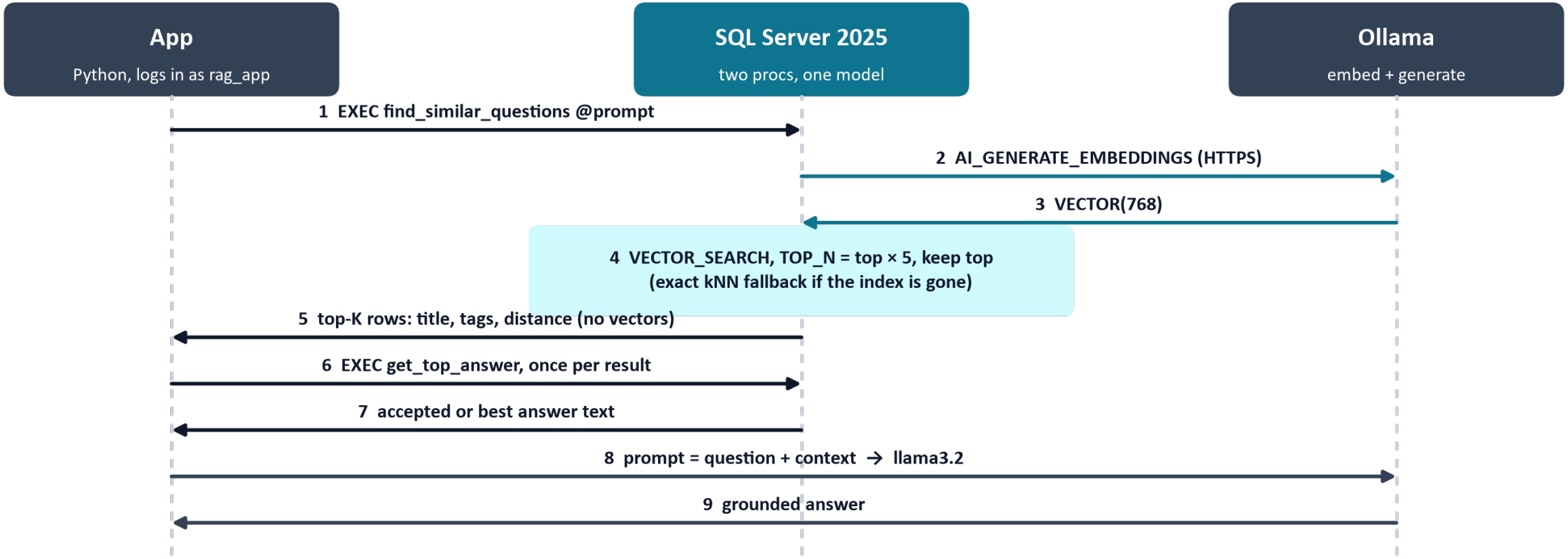
The test lab

One laptop: SQL Server 2025 CU10 in Docker, Ollama on the host, no cloud



RAG application architecture

The app sends text and gets rows back. It never sees a vector.



rag_app has EXECUTE on two procs and the model, and no SELECT on any table. The app never sees a vector.

Patterns that make it production-ready

What Demo 3 builds, and why

Satellite table

Vectors in their own table and filegroup.

- Single INT key for the index
- Rebuild without touching Posts
- chunk = what was embedded

Procs as the contract

The app calls two procs.

- Exact fallback with no index
- Oversamples 5× for recall
- Swap models in one place

Least privilege

rag_app can EXECUTE, not SELECT.

- VIEW DEFINITION so the proc sees the index
- No access to the vectors
- Default database: StackOverflow

Operate it

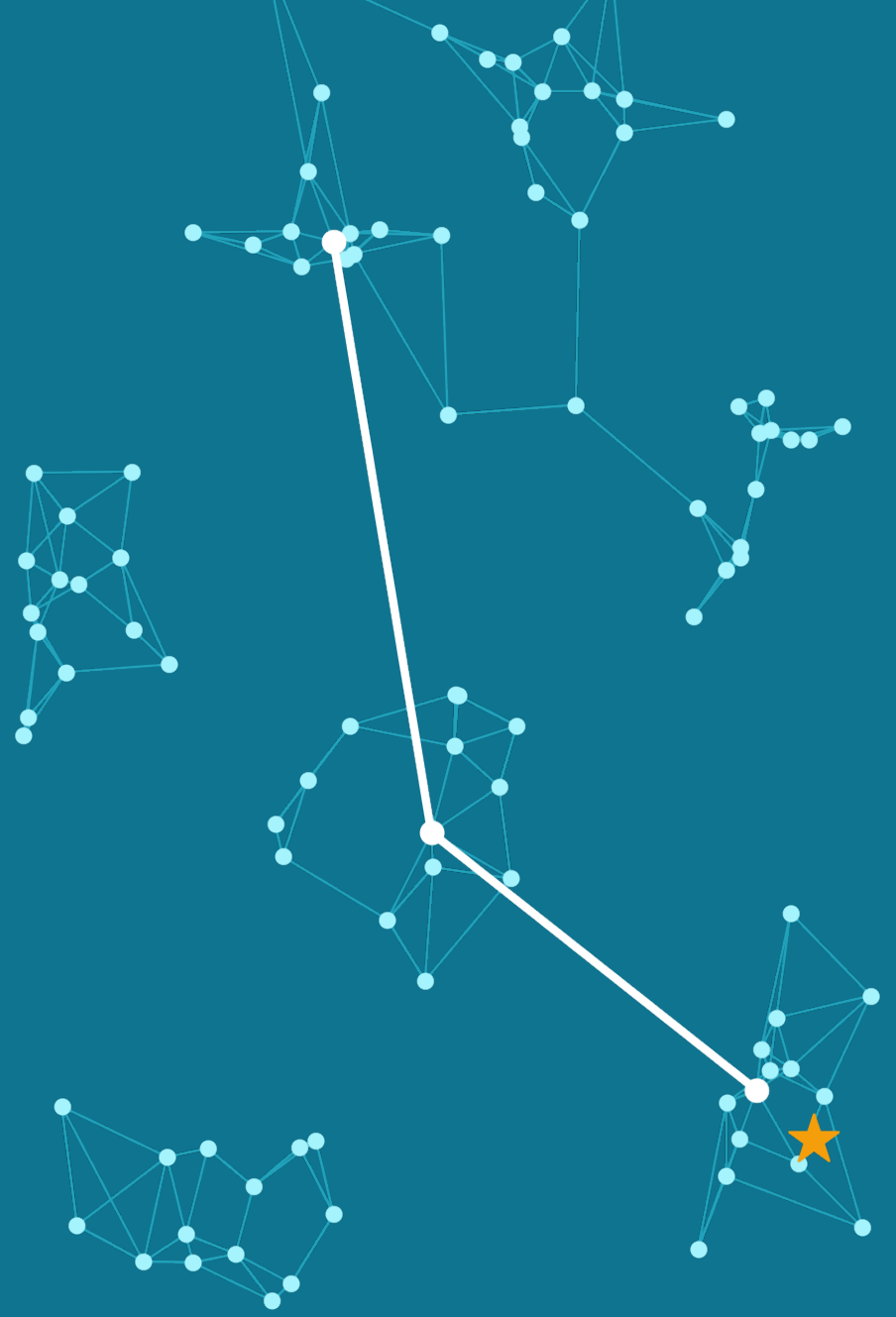
Drop, load, rebuild on a schedule.

- The fallback isn't free at 1M rows
- Check for stale chunks
- Alert when the app reports kNN

LIVE DEMO

Demo 3: The RAG application

Schema and chunking · the search proc · least privilege · drop, load, rebuild · the app



Takeaways

Measure on your data, your model, your hardware

Storage

A VECTOR is a native float array in the row.

- float16 halves it for ~0.00001 of distance
- Embedding costs ~15 ms a row*

Indexes

Know where the line is.

- Exact search under ~50,000 vectors
- ANN: ~70× fewer reads at 1M rows
- Oversample to keep recall

Production

Put it behind procs.

- Least privilege by default
- Plan for read-only tables on 2025
- Watch the fallback path

** Environment specific: nomic-embed-text on Ollama, Apple M2 GPU, through a local NGINX TLS proxy. Your model, hardware, and network will differ, so measure it.*

Questions?

github.com/nocentino/vector-under-the-hood

Every demo, the lab, and the app, running on your laptop

- Vector data type: learn.microsoft.com/sql/t-sql/data-types/vector-data-type
- CREATE VECTOR INDEX: learn.microsoft.com/sql/t-sql/statements/create-vector-index-transact-sql
- VECTOR_SEARCH: learn.microsoft.com/sql/t-sql/functions/vector-search-transact-sql
- DiskANN: Subramanya et al., NeurIPS 2019